

RAG in Studio

LESSON 01 · KNOWLEDGE & RAG

What you'll learn

- Explain why a plain model answers business-specific questions confidently but wrongly, and what grounding fixes
- Describe the retrieve-then-answer loop that turns an attached knowledge base into a sourced answer
- Read a RAG retrieval step inside the trace timeline to verify exactly which content an answer came from
- Recognise how an agent draws from multiple attached knowledge bases based on what the question needs

Key concepts

A model only knows what it was trained on — it doesn't know your prices, your policies, or your product. Ask a plain model about your business and it will answer with total confidence and be completely wrong. The entire Knowledge track exists to fix that single gap: grounding the agent in your own knowledge instead of its training data.

RAG — retrieval-augmented generation — means the agent retrieves the most relevant pieces of your knowledge before it writes, then answers from those. When the Northwind support agent is asked for the laptop warranty policy, it pulls the exact terms from the attached support docs and answers from them: a one-year limited manufacturer warranty covering defects in materials and workmanship, excluding accidental, liquid, and wear damage. Every bit of that came straight from the documents, not the model's imagination.

A trace lets you prove the answer came from your documents, not from guesswork. Open the traces for a message, go into the trace timeline, and click the RAG retrieval step — you see exactly what the agent searched for and the content it pulled back from the knowledge base to build the answer. That verifiability is what separates an agent that genuinely knows your business from one that just sounds like it does.

Attach more than one knowledge base and the agent pulls from whichever ones are relevant to the question. You're not locked to a single source — Studio routes the retrieval to the knowledge that actually matters for each query, so a support agent and a policy library can coexist behind one conversation.

In Studio

Primary surface: **Knowledge → Knowledge Base**, with verification in the agent's **Traces**.

TAKEAWAY

A model only knows what it was trained on — it doesn't know your prices, your policies, or your product.

TAKEAWAY

RAG — retrieval-augmented generation — means the agent retrieves the most relevant pieces of your knowledge before it writes, then answers from those.

TAKEAWAY

A trace lets you prove the answer came from your documents, not from guesswork.

TAKEAWAY

Attach more than one knowledge base and the agent pulls from whichever ones are relevant to the question.

This lesson leans on a knowledge base built earlier (you build one from scratch in the next lesson), so here the focus is seeing RAG work and proving it:

1. Open the agent that already has a knowledge base attached — here, the **Northwind support agent** with the support docs attached as its knowledge base.
2. In the chat, ask something specific to the business, e.g. *"What's the warranty policy for laptops at Northwind?"*
3. Read the answer — it returns the warranty terms drawn straight from the documents, and even offers to pull the terms for monitors or accessories.
4. Open **Traces** for that message and go into the **trace timeline**.
5. Click the **RAG retrieval** step to see exactly what the agent searched for and the content it pulled back from the knowledge base to build the answer.
6. (When applicable) attach additional knowledge bases to the agent — it will retrieve from whichever ones are relevant to each question.

TRY THIS

- 1** Take an agent with a knowledge base attached and ask it a question whose answer lives only in those documents. Then ask the same model the question with no knowledge base — compare how confident, and how wrong, the ungrounded answer is.
- 2** Open the **Traces** for that grounded answer, find the **RAG retrieval** step in the trace timeline, and read the retrieved chunks. Confirm that every claim in the answer traces back to a pulled passage — and flag any sentence that doesn't.
- 3** Attach a second knowledge base covering a different topic, then ask a question that only the new source can answer. Check the trace to confirm the agent retrieved from the right knowledge base.